Finite State Machine Implementation

Joel Vincent

1. Introduction

The purpose of this project was to implement and demonstrate a finite state machine using an Arduino Nano. Embedded systems are often reactive systems in which the current state of the device and its inputs determine how the system behaves. A finite state machine provides a structured method of representing this behavior through a set of states and transitions.

The state machine consisted of four states: dark, red, blue, and green. Each state corresponded to a color displayed using the LED on the Arduino. Transitions between states occurred either through a switch input or after a specified amount of time. Because the provided board does not have a built-in button, a soft switch was implemented using serial communication between the Arduino and a laptop. Entering the character s or S through the Arduino IDE Serial Monitor represented pressing the switch.

The completed system successfully used the LED, Arduino board, serial input, and software timing to reproduce the behavior described by the provided state diagram.

2. Hardware Design

The primary hardware component used for this project was an Arduino Nano 33 BLE Sense Rev2. The board was connected to a laptop through USB for programming, power, and serial communication. No additional external hardware was required because the built-in RGB LED was used as the system output and the laptop provided the switch input.

The built-in RGB LED is connected to pins 22, 23, and 24. These were assigned to the red, green, and blue components of the LED, respectively. Each pin was configured as an output during the setup() function.

Soft Switch Implementation

Since the development board does not provide the physical switch required by the state diagram, a soft switch was created using the Serial Monitor. During execution, the program continually checks whether data is available on the serial connection. When ‘s’ is pressed, the program interprets the input as one press of SW.

3. Software Design and State Machine Implementation

The four states were represented in the program using an enumerated data type:

DARK, RED, BLUE, and GREEN.

The variable currentState stores the current operating state of the system. The initial value is DARK, satisfying the requirement that the system begins with the LED turned off.

The main loop() performs two tasks. First, it checks the serial connection for a simulated switch press. Second, a switch statement examines the current state and determines whether a transition should occur.

Table of states and transitions

Current State	Event	Next State
Dark	SW	Red
Red	5 sec timeout	Dark
Red	SW	Blue
Blue	4 sec timeout	Red
Blue	SW	Green
Green	3 sec timeout	Blue
Green	SW	Dark

A separate changeState() function performs the actions associated with entering a new state. The function updates currentState, records the time at which the new state begins, changes the LED to the corresponding color, and prints the new state to the Serial Monitor. Separating the state-transition logic from the LED control helped keep the main state machine organized.

```
48    switch (currentState) {
49      case DARK:
50        if (switchPressed) {
51          changeState(RED);
52        }
53        break;
54
55      //I'm using millis here instead of delay because delay will ignore live inputs from my keyboard
56      //If I used delays, then it would just be stuck flipping between red and dark
57      case RED:
58        // SW has priority over timeout
59        if (switchPressed) {
60          changeState(BLUE);
61        } else if (millis() - stateStartTime >= 5000) { //5 sec
62          changeState(DARK);
63        }
64        break;
65
66      case BLUE:
67        // SW has priority over timeout
68        if (switchPressed) {
69          changeState(GREEN);
70        } else if (millis() - stateStartTime >= 4000) { //4 sec
71          changeState(RED);
72        }
73        break;
74
75      case GREEN:
76        // SW has priority over timeout
77        if (switchPressed) {
78          changeState(DARK);
79        } else if (millis() - stateStartTime >= 3000) { //3 sec
80          changeState(BLUE);
81        }
82        break;
83    }
```

```
96    void changeState(State newState) {
97      currentState = newState;
98      stateStartTime = millis();
99
100     switch (currentState) {
101       case DARK:
102         SetColor(255, 255, 255);
103         Serial.println("State: DARK");
104         break;
105
106       case RED:
107         SetColor(0, 255, 255);
108         Serial.println("State: RED");
109         break;
110
111       case BLUE:
112         SetColor(255, 255, 0);
113         Serial.println("State: BLUE");
114         break;
115
116       case GREEN:
117         SetColor(255, 0, 255);
118         Serial.println("State: GREEN");
119         break;
120     }
121   }
```

4. Timing Design

An important part of the project was implementing the automatic transitions from Red to Dark, Blue to Red, and Green to Blue. These transitions occur after five, four, and three seconds, respectively.

I originally wanted to use the delay() function. Although delay() could produce the required amount of waiting time, it created a problem for the state machine. While the processor was executing a delay, the main program was not able to return to the serial-input checking code. As a result, a switch command sent during the waiting period was not immediately processed. This prevented the system from correctly responding to both timed and user-generated events. In practice, I couldn't even get past red. It would constantly cycle between red and dark states.

To solve this problem, non-blocking timing was implemented using millis(). The variable stateStartTime records the value of millis() whenever the system enters a new state. The program can then determine the elapsed time using millis() - stateStartTime.

5. Observations and Results

The final implementation successfully reproduced the behavior of the provided state machine. The system initialized in the dark state with the RGB LED off. Sending the soft switch command caused the appropriate transition based on the current state, while leaving the system without input resulted in the expected timed transitions from red, blue, and green.

6. Challenges

The primary challenge I came across involved implementing the timed state transitions without preventing the system from detecting switch inputs. I wanted to use the delay() function for the 5, 4, 3 second delays. However, testing showed that the system did not respond correctly when an s command was sent while a delay was active. It would flip between the red and dark states.

This issue was fixed by replacing the blocking delays with elapsed-time comparisons using millis(). This required storing the time at which each state began and repeatedly comparing the current time against that value. This allowed the state machine to continue processing inputs while waiting for a timeout.

7. Conclusion

A four-state finite state machine was successfully implemented on the Arduino Nano. The built-in LED represents the dark, red, blue, and green states, while a serial soft switch lets us have an input, since the board does not have one built in. Timed transitions were implemented using millis() so that the system could remain responsive to switch commands while tracking elapsed time.

Seven test cases were used to evaluate all seven transitions of the provided state diagram. The final implementation produced the expected state and LED output for each tested transition. The project demonstrated how a state-machine model can showcase a functioning embedded system.